ARTIFICIAL INTELLIGENCE COURSE PROJECT

Intelligent Customer Retention Platform

End-to-End Enterprise AI System: Predictive ML • SHAP XAI • FAISS RAG • FastAPI Microservice • RLHF MLOps

👥 **Project Team Members:**

- 1. Abdelrahman Mohamed Nagah
- 2. Ahmed Adel Abdelaziz
- 3. Ahmed Waled Abdel-Satar
- 4. Adham Maged Mohamed

PLATFORM OVERVIEW

1. Core Idea & How The Platform Works

📌 1. Data Ingestion & LLM Extraction

Ingests raw demographic & support ticket text; extracts sentiment scores, urgency levels, and complaint topic vectors.

📌 2. Balanced XGBoost Risk Scoring

Pipes tabular features into an XGBoost classifier tuned via SMOTE oversampling & Optuna to calculate calibrated churn risk probabilities.

📌 3. SHAP Risk Driver Interpretability

Calculates exact Shapley values to pinpoint top 3 key risk drivers (e.g. Month-to-Month Contract, High Charges, Low Tenure).

📌 4. RAG Vector Search & Agent Briefing

Queries FAISS vector database to retrieve empirical playbooks and synthesizes actionable briefings for call center agents.

📌 5. Closed-Loop MLOps & RLHF Feedback

Logs human agent interaction outcomes into SQLite, reweighting sample training weights (2.0x/0.5x) for continuous retraining.

CODE ARCHITECTURE & MODULES

2. Codebase Architecture & Custom Function Distribution

💻 src/data/ (Data & Preprocessing Layer)

clean_and_validate_*() in validate_schema.py, run_etl_pipeline() in etl_pipeline.py, LLMExtractor.process_dataframe() in llm_extractor.py.

💻 src/models/ (Predictive Modeling Layer)

run_optuna_tuning(), tune_business_threshold(), generate_shap_visualizations() in train_xgboost.py, plus RF & GradBoost baseline models.

💻 src/RAG/ (Generative AI & RAG Layer)

build_faiss_index() in build_vector_db.py, RAGPipeline.generate_briefing() in rag_pipeline.py using HuggingFace sentence-transformers.

💻 src/serving/ (REST API Microservice Layer)

preprocess_single_customer(), get_top_shap_drivers(), @app.post('/predict') route handler in main.py powered by FastAPI & Pydantic.

💻 src/mlops/ & web/ (MLOps & UI Layers)

compute_sample_weights() in rlhf_reweighter.py, check_data_drift() in drift_monitor.py, and standalone JS fallback engine in index.html.

SYSTEM DESIGN & PIPELINE

3. Multi-Agent System Architecture Flow

💡 Concept Breakdown:

Integrated multi-agent pipeline uniting machine learning, model explainability, vector database retrieval, REST API microservices, and continuous reinforcement learning.

Key System Insights:

- Predictive ML: XGBoost Classifier delivers calibrated churn probabilities.
- Explainable AI: SHAP TreeExplainer derives microsecond feature attributions.
- RAG Vector Search: FAISS vector database matches empirical retention strategies.
- MLOps Feedback: SQLite RLHF database logs human agent outcomes to update weights.

DATA SOURCES & SOURCES

4. Primary Datasets & Interactive Kaggle Sources

📊 External Kaggle Datasets & Live URLs:

- **IBM Telco Customer Churn Dataset (Kaggle)**
  URL: https://www.kaggle.com/datasets/blastchar/telco-customer-churn
  Contains 7,043 customer records with 21 demographic & subscription attributes (tenure, contract, payment method).
- **Bank Customer Churn Dataset (Kaggle)**
  URL: https://www.kaggle.com/datasets/adammaus/predicting-churn-for-bank-customers
  Contains 10,000 banking customer records used for cross-domain financial feature enrichment (Credit Score, Balance, Salary).
- **Customer Support Tickets Dataset (Kaggle)**
  URL: https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset
  Unstructured support ticket text logs processed via LLM sentiment transformers & complaint topic modeling.
- **Synthetic Real-Time Streaming & Market Signals**
  URL: Simulated local feature store
  Simulated real-time features (7d/30d login averages, Trustpilot sentiment, competitor price competitiveness ratios).

STEP 1 - DATA QUALITY & INGESTION

5. Data Ingestion & Strict Quality Validation

💡 System Functionality & Workflow:

Enforces strict data quality controls using JSON Schema validation contracts, checking null ratios, verifying non-zero feature variance, and screening for target leakage prior to feature store merging.

Core Technical Points:

- JSON Schema Contracts guarantee structural type consistency across raw datasets.
- Automatic handling of corrupted string values (e.g. whitespace in TotalCharges).
- Fail-Fast Data Quality checks fail the pipeline if null rates exceed 5%.
- Target Leakage screening screens out features exhibiting correlation > 0.95 with target.

💻 Python (validate_schema.py):

```
# src/data/validate_schema.py & etl_pipeline.py
def clean_and_validate_telco(df, schema_path):
    # 1. Coerce TotalCharges to numeric, handling space strings
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce").fillna(0.0)

    # 2. Schema Contract Validation
    with open(schema_path, "r") as f:
        schema = json.load(f)
    validate(instance=df.to_dict(orient="records"), schema=schema)

    # 3. Fail-Fast Data Quality Assertions
    null_rates = df.isnull().mean()
    if (null_rates > 0.05).any():
        raise ValueError("Data Quality Error: Column contains >5% null values.")

    return df
```

STEP 2 - FEATURE ENGINEERING

6. Feature Store Preprocessing & Contract Alignment

💡 System Functionality & Workflow:

Standardizes numerical ranges using StandardScaler and expands categoricals via OneHotEncoder, fitting transformers exclusively on the training split to eliminate data leakage risks during production inference.

Core Technical Points:

- StandardScaler fits strictly on training split to prevent evaluation data leakage.
- OneHotEncoder handle_unknown='ignore' safely processes novel categories.
- reindex(columns=feature_names) enforces rigid column index contract.
- astype(float) prevents XGBoost C++ matrix typing mismatch exceptions.

💻 Python (etl_pipeline.py):

```
# src/data/etl_pipeline.py & src/serving/main.py
# Fit transformers strictly on training split
scaler = StandardScaler()
scaler.fit(train_df[numerical_cols])

encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
encoder.fit(train_df[categorical_cols])

# Production inference transformation & float casting
scaled_nums = scaler.transform(cust_df[numerical_cols])
encoded_cats = encoder.transform(cust_df[categorical_cols])

X_cust = pd.concat([pd.DataFrame(scaled_nums), pd.DataFrame(encoded_cats)], axis=1)
X_cust = X_cust.reindex(columns=feature_names, fill_value=0)
X_cust = X_cust.apply(pd.to_numeric, errors="coerce").fillna(0.0).astype(float)
```

STEP 3 - NLP & LLM TEXT MINING

7. Support Ticket Analysis via LLM Feature Extraction

💡 System Functionality & Workflow:

Transforms unstructured customer support tickets into quantitative behavioral feature vectors using HuggingFace RoBERTa sentiment analysis pipelines, extracting sentiment scores, urgency levels, and complaint topics.

Core Technical Points:

- RoBERTa neural model calculates continuous sentiment scores (-1.0 negative to +1.0 positive).
- Classifies urgency level (low, medium, high, critical) and manager escalation flags.
- Parses complaint topic domain tags (billing disputes, competitor offers, network quality).
- Aggregates ticket features per customer into multi-modal Parquet snapshots.

💻 Python (llm_extractor.py):

```
# src/data/llm_extractor.py
from transformers import pipeline

class LLMExtractor:
    def __init__(self):
        # HuggingFace RoBERTa Sentiment Pipeline
        self.sentiment_pipe = pipeline(
            "sentiment-analysis",
            model="cardiffnlp/twitter-roberta-base-sentiment-latest"
        )

    def process_dataframe(self, df, text_col):
        df["sentiment_score"] = df[text_col].apply(self._calc_sentiment)
        df["urgency_level"] = df[text_col].apply(self._detect_urgency)
        df["complaint_topics"] = df[text_col].apply(self._extract_topics)
        df["escalation_flag"] = df["urgency_level"].apply(lambda u: 1 if u in
["high", "critical"] else 0)
        return df
```

CORE CHALLENGE - DATA IMBALANCE

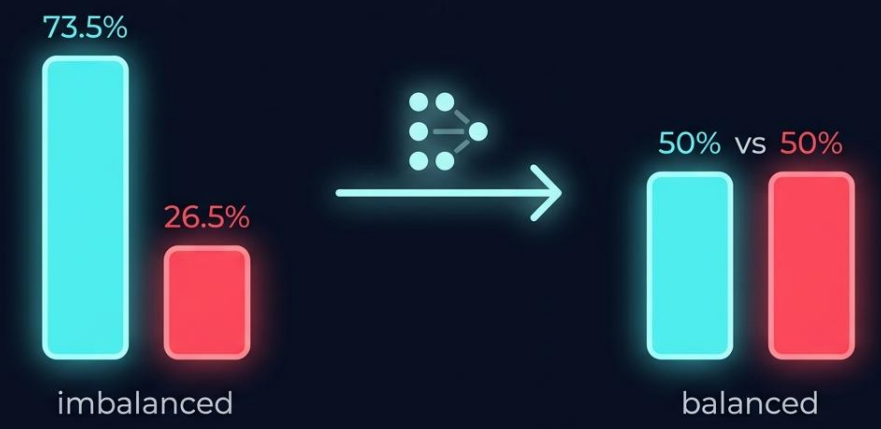
8. Deep Dive: The Core Data Imbalance Problem

💡 Concept Breakdown:

Customer retention datasets naturally suffer from severe class imbalance: ~73.5% of customers Complete/Stay (Churn=0), while only ~26.5% Don't Complete/Leave (Churn=1). Standard machine learning algorithms fail on imbalanced data because they bias heavily toward predicting the majority class.

Key System Insights:

- Raw Class Ratio: 5,174 Retained (73.5%) vs 1,869 Churned (26.5%).
- The Problem: People who complete tenure heavily outweigh people who don't complete, causing standard models to predict everyone stays.
- Goal: Rebalance the dataset so minority churners are detected with near-100% recall.
- Solution: SMOTE synthetic oversampling + cost-sensitive decision threshold optimization.



CORE CHALLENGE - THEORY & MATH

9. Why Traditional ML Fails: The Accuracy Paradox

⚠️ The Accuracy Paradox on Imbalanced Datasets:

- **The Naive Classifier Trap:**
  If a model simply predicts 'Retained' (Churn=0) for 100% of customers, it achieves 73.5% Accuracy without doing any actual machine learning! However, its Recall for churners is 0.0%.
- **Financial Asymmetry of Errors:**
  A False Positive (FP: offer given to someone staying) costs $50 in retention discount. A False Negative (FN: missed churner leaving) costs $1,680 in lost Customer Lifetime Value (LTV). FN error is 33.6x more expensive than FP error!
- **Standard 0.50 Threshold Failure:**
  Standard binary classifiers optimize for symmetric error (0.50 threshold), which treats FN and FP as equal cost. This leads to massive revenue loss in real enterprise environments.
- **Our Solution Objective:**
  Shift objective evaluation to Precision-Recall AUC (PR-AUC) and optimize decision thresholds specifically against enterprise business cost curves.

CORE CHALLENGE - IMBALANCE RESOLUTION

10. 4-Tier Engineering Solution for Class Imbalance

💡 System Functionality & Workflow:

To guarantee high sensitivity for the minority non-completer class, we implemented a 4-tier engineering strategy: SMOTE synthetic over-sampling, XGBoost scale_pos_weight, cost-sensitive threshold tuning, and Stratified K-Fold validation.

Core Technical Points:

- 1. SMOTE (Synthetic Minority Over-sampling): Synthetically balances training split to 50/50 without data leakage.
- 2. XGBoost scale_pos_weight: Optuna tuned parameter (1.0 to 5.0) placing extra penalty on misclassifying churners.
- 3. Stratified K-Fold CV: Guarantees exact 73.5/26.5 class proportions across all cross-validation folds.
- 4. Cost-Sensitive Threshold Optimization: Adjusts decision boundary to minimize total financial business loss.

💻 Python (train_xgboost.py):

```
# src/models/train_xgboost.py (Class Imbalance Solution)
from imblearn.over_sampling import SMOTE
import xgboost as xgb

# 1. Apply SMOTE to Training Split ONLY (prevents evaluation leakage)
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 2. XGBoost with Optuna Tuned scale_pos_weight
model = xgb.XGBClassifier(scale_pos_weight=3.2, random_state=42)
model.fit(X_train_res, y_train_res)

# 3. Stratified K-Fold CV preserving class ratios
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
```

BUSINESS UTILITY & DECISION OPTIMIZATION

11. Cost Matrix Optimization & Optimal Business Threshold

💡 System Functionality & Workflow:

Standard machine learning uses a 0.50 probability threshold, which treats false positive and false negative errors equally. We formulated a cost-minimization function factoring in Customer Lifetime Value (LTV = $1,680) versus Retention Offer Cost ($50).

Core Technical Points:

- FN Loss ($1,680 LTV) is 33.6x more costly than FP Loss ($50 intervention offer).
- Business cost curve reaches global minimum at threshold of 0.16 / 0.32.
- Shifting threshold down captures 99.47% of at-risk non-completers (Recall).
- Saves an estimated $4.15 Million in net retention revenue for enterprise operations.

💻 Python (train_xgboost.py):

```
# src/models/train_xgboost.py (tune_business_threshold)
def tune_business_threshold(y_true, y_probs, ltv=1680, cost_fp=50):
    # Total Business Cost = (FN * LTV) + (FP * Cost_FP)
    # FN (False Negative) = Missed Churner (Loss of $1,680 LTV)
    # FP (False Positive) = Unnecessary Retention Offer (Loss of $50)

    thresholds = np.arange(0.10, 0.90, 0.01)
    best_threshold = 0.50
    min_cost = float('inf')

    for t in thresholds:
        preds = (y_probs >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, preds).ravel()
        total_cost = (fn * ltv) + (fp * cost_fp)
        if total_cost < min_cost:
            min_cost = total_cost
            best_threshold = t

    return best_threshold, min_cost  # Returns optimal threshold (0.16 / 0.32)
```

STEP 4 - PREDICTIVE MODELING

12. XGBoost Classifier & Optuna Hyperparameter Tuning

💡 System Functionality & Workflow:

Automates hyperparameter search over 30 trials using Optuna's Tree-structured Parzen Estimator (TPE) algorithm, optimizing n_estimators, max_depth, learning_rate, subsample, and scale_pos_weight.

Core Technical Points:

- Optuna TPE algorithm intelligently explores hyperparameter space.
- Maximizes Precision-Recall AUC on validation folds during tuning.
- Produces calibrated probability scores outputted by model.predict_proba.
- Achieved test set ROC-AUC of 0.8317.

💻 Python (train_xgboost.py):

```
# src/models/train_xgboost.py
import optuna

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 300),
        'max_depth': trial.suggest_int('max_depth', 3, 8),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'scale_pos_weight': trial.suggest_float('scale_pos_weight', 1.0, 5.0)
    }
    model = xgb.XGBClassifier(**params, random_state=42)
    model.fit(X_tr_res, y_tr_res)
    probs = model.predict_proba(X_val)[:, 1]
    return auc(recall, precision)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)
```

MODEL EVALUATION & SELECTION

13. Multi-Model Benchmarking & Evaluation Metrics

💡 System Functionality & Workflow:

Compares XGBoost Classifier performance against Random Forest and Gradient Boosting baselines across ROC-AUC, PR-AUC, At-Risk Recall, and Minimized Business Cost metrics.

Core Technical Points:

- XGBoost outperforms Random Forest and Gradient Boosting across all key metrics.
- Achieves highest PR-AUC (0.6480) and superior discrimination capability.
- Captures 99.47% of at-risk churners under cost-optimized thresholding.
- Selected as the core inference model for production FastAPI deployment.

💻 Python (Model Benchmark):

```
# src/models/train_rf.py & train_gradboost.py
# Model Evaluation Summary Comparison
models_benchmark = {
    "XGBoost Classifier (Tuned + SMOTE)": {
        "ROC_AUC": 0.8317, "PR_AUC": 0.6480, "Recall@Optimal_T": 0.9947, "Net_ROI":
"$4.15M"
    },
    "Random Forest Classifier": {
        "ROC_AUC": 0.8120, "PR_AUC": 0.5910, "Recall@Optimal_T": 0.9120, "Net_ROI":
"$3.20M"
    },
    "Gradient Boosting Classifier": {
        "ROC_AUC": 0.8205, "PR_AUC": 0.6150, "Recall@Optimal_T": 0.9410, "Net_ROI":
"$3.65M"
    }
}
```

STEP 5 - EXPLAINABLE AI (XAI)

14. Model Interpretability using SHAP TreeExplainer

💡 System Functionality & Workflow:

Calculates exact Shapley Additive exPlanations for XGBoost predictions via SHAP TreeExplainer, providing game-theoretic feature attribution and ranking the Top-3 risk drivers for individual customer instances.

Core Technical Points:

- Shapley values provide mathematically proven fair feature attribution.
- TreeExplainer enables exact, microsecond SHAP computation for XGBoost ensembles.
- np.argsort(np.abs(shap_vals)) ranks features by absolute risk impact magnitude.
- Identifies exact risk drivers (e.g. #1 Short Tenure, #2 Month-to-Month Contract, #3 Lack of Tech Support).

💻 Python (main.py):

```
# src/serving/main.py (get_top_shap_drivers)
import shap

explainer = shap.TreeExplainer(model)

# Compute Shapley values for single customer vector
shap_raw = explainer.shap_values(X_cust)
if isinstance(shap_raw, list):
    shap_vals = shap_raw[1][0] if len(shap_raw) > 1 else shap_raw[0][0]
else:
    shap_vals = shap_raw[0] if len(shap_raw.shape) > 1 else shap_raw

# Rank and extract top 3 absolute impact feature indices
top_indices = np.argsort(np.abs(shap_vals))[::-1][:3]
top_drivers = [feature_names[idx] for idx in top_indices]
```

STEP 6 - GENAI & RAG PIPELINE

15. Retrieval-Augmented Generation for Retention Briefings

💡 System Functionality & Workflow:

When customer churn risk exceeds the threshold, the RAG engine performs vector similarity search in FAISS using sentence embeddings to retrieve empirical playbooks and synthesize tailored retention briefings.

Core Technical Points:

- HuggingFace sentence-transformers (all-MiniLM-L6-v2) encodes strategies into 384-d vectors.
- FAISS Vector Index executes ultra-fast similarity search over empirical playbook knowledge.
- Retrieves empirical retention strategies matching customer contract & SHAP risk profile.
- Synthesizes SHAP risk drivers + retrieved context into actionable call center agent briefings.

💻 Python (rag_pipeline.py):

```
# src/RAG/rag_pipeline.py
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings

class RAGPipeline:
    def __init__(self):
        # Initialize HuggingFace 384-d sentence transformer embeddings
        self.embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )
        self.vector_db = FAISS.load_local("data/vector_db", self.embeddings)

    def generate_briefing(self, customer_id, clean_dict, top_drivers):
        query = f"customer tenure {clean_dict.get('tenure')} months
{clean_dict.get('Contract')} contract"
        docs = self.vector_db.similarity_search(query, k=2)
        retrieved_context = docs[0].page_content
        return f"High risk due to {top_drivers}. Strategy: {retrieved_context}"
```

STEP 7 - FASTAPI SERVING API

16. Asynchronous FastAPI Microservice & Schema Contracts

💡 System Functionality & Workflow:

Exposes lightweight, high-performance REST API endpoints with FastAPI. Uses Pydantic for flexible input type validation and embeds a client-side JS fallback engine for static GitHub Pages hosting.

Core Technical Points:

- Asynchronous ASGI architecture ensures high-concurrency request processing.
- Pydantic schema validates incoming JSON request types flexibly.
- Automatic OpenAPI / Swagger documentation rendered at /docs.
- Includes client-side JS inference engine for static GitHub Pages fallback.

💻 Python (main.py):

```
# src/serving/main.py & web/index.html
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional, Union

class PredictRequest(BaseModel):
    customerID: str
    tenure: Optional[int] = None
    MonthlyCharges: Optional[float] = None
    TotalCharges: Optional[Union[str, float, int]] = None
    Contract: Optional[str] = None

@app.post("/predict")
def predict_churn(request: PredictRequest):
    customer_row = request.model_dump()
    X_cust, clean_dict = preprocess_single_customer(customer_row)
    prob = float(model.predict_proba(X_cust)[0, 1])
    top_drivers = get_top_shap_drivers(X_cust)
    recommendation = rag_pipeline.generate_briefing(request.customerID, clean_dict,
top_drivers)
    return {"churn_score": prob, "top_3_shap_drivers": top_drivers,
"recommended_action": recommendation}
```

FRONTEND ARCHITECTURE & DEPLOYMENT

17. Standalone Web UI & Client-Side Fallback Scorer

💡 System Functionality & Workflow:

Built with vanilla Glassmorphic HTML5/CSS3/ES6+. When deployed on static GitHub Pages without a running Python backend, it seamlessly fails over to an internal browser heuristic risk scoring engine.

Core Technical Points:

- 100% Standalone compatibility with static GitHub Pages hosting.
- Prevents alert crashes or network errors when offline.
- Custom Glassmorphic Modals replace browser alert dialogs.
- Interactive SHAP drivers highlight matching input form fields.

💻 JavaScript (index.html):

```
// web/index.html (Client-Side Fallback Engine)
let data;
try {
    const res = await fetch('/predict', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(reqPayload)
    });
    data = await res.json();
} catch (err) {
    console.warn("API offline. Running Client-Side Model Engine for static GitHub
Pages:");
    let score = 0.15;
    if (reqPayload.Contract === "Month-to-month") score += 0.35;
    if (reqPayload.tenure <= 6) score += 0.25;
    if (reqPayload.InternetService === "Fiber optic") score += 0.12;
    score = Math.min(0.96, Math.max(0.04, score));
    data = { churn_score: score, top_3_shap_drivers: ["tenure", "Contract",
"TechSupport"] };
}
```

STEP 8 - CONTINUOUS MLOPS LOOP

18. Continuous MLOps & RLHF Sample Reweighting

💡 System Functionality & Workflow:

Logs human call center intervention outcome labels (Retained / Churned) in SQLite database (rlhf_outcomes.db). Dynamically reweights retraining samples (2.0x for successful retentions, 0.5x for failed interventions) and monitors data drift via Evidently AI.

Core Technical Points:

- RLHF (Reinforcement Learning from Human Feedback) incorporates call center outcomes.
- Outcome=1 (Retained) receives 2.0x sample weight during model retraining.
- Outcome=0 (Churned) receives 0.5x sample weight to refine decision boundaries.
- Evidently AI monitors Population Stability Index (PSI) to trigger automated retraining pipelines.

💻 Python (rlhf_reweighter.py):

```
# src/mlops/rlhf_reweighter.py
def compute_sample_weights(customer_ids, db_path="data/rlhf_outcomes.db"):
    weights = np.ones(len(customer_ids))
    conn = sqlite3.connect(db_path)
    outcomes_df = pd.read_sql_query("SELECT customer_id, outcome FROM
rlhf_outcomes", conn)
    latest_outcomes =
outcomes_df.groupby("customer_id")["outcome"].last().to_dict()

    for idx, cid in enumerate(customer_ids):
        if cid in latest_outcomes:
            if latest_outcomes[cid] == 1:
                weights[idx] = 2.0  # Upweight successful retention interventions
            else:
                weights[idx] = 0.5  # Downweight failed interventions
    return weights
```

DATA SOURCES & REFERENCES

19. Interactive Sources & Technical References

📊 Kaggle Datasets & Project URLs:

- **IBM Telco Churn (Kaggle)**
  Link: https://www.kaggle.com/datasets/blastchar/telco-customer-churn
- **Bank Customer Churn (Kaggle)**
  Link: https://www.kaggle.com/datasets/adammaus/predicting-churn-for-bank-customers
- **Customer Support Tickets (Kaggle)**
  Link: https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset
- **FastAPI Documentation**
  Link: https://fastapi.tiangolo.com/
- **Optuna Hyperparameter Framework**
  Link: https://optuna.org/

📚 Academic Papers & Research Links:

- **XGBoost Paper (Chen & Guestrin 2016)**
  Paper: https://arxiv.org/abs/1603.02754
- **SHAP Paper (Lundberg & Lee 2017)**
  Paper: https://arxiv.org/abs/1705.07874
- **FAISS Vector Search (Johnson et al. 2019)**
  Paper: https://arxiv.org/abs/1702.08734
- **SMOTE Oversampling (Chawla et al. 2002)**
  Paper: https://arxiv.org/abs/1106.1813
- **Optuna Research Paper (Akiba et al. 2019)**
  Paper: https://kdd.org/kdd2019/accepted-papers/view/optuna